

M5 + M6 — What this 15-minute block gets you

- ✓ M5 in one line— the escape hatch when SQL is not enough: custom code, streaming, non-BigQuery targets
- ✓ Batch vs streaming as a latency decision— Pub/Sub, Dataflow and templates, named not drilled
- ✓ M6 = your CI/CD analogue— Cloud Scheduler $\approx$ SQL Agent, Composer $\approx$ the orchestrator
- ✓ Aimed at the M&O team— who keeps jobs running: scheduling, retries, failure visibility
- ✓ No lab in this block— the Spark lab is optional homework; the close schedules one live query

Last block of Day 1 — no lab, then we close

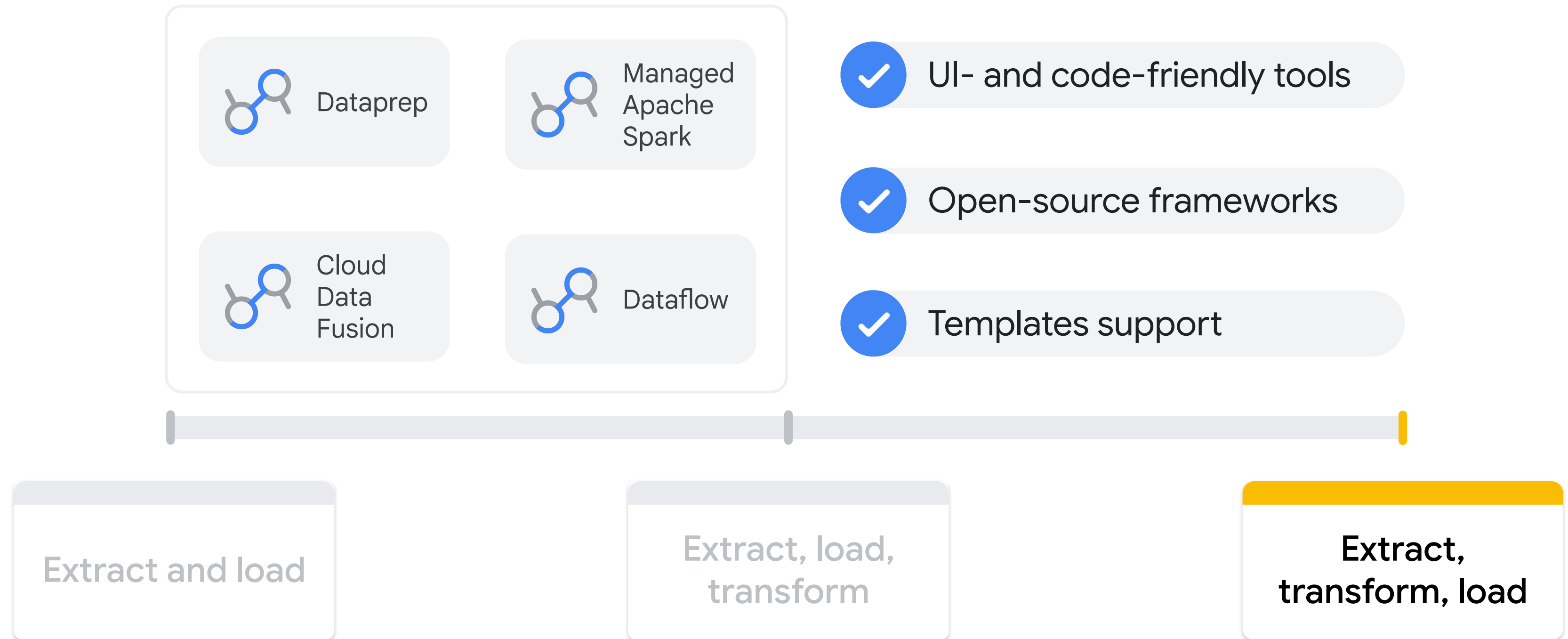
16:05–16:15 M5 compressed — ETL options: batch vs streaming, Pub/Sub, Dataflow, templates

16:15–16:20 M6 compressed — automation: Scheduler, Composer, Dataform release configs

16:20–16:30 Close: schedule the derived-table query live, Day 1 recap, Day 2 preview

DEMO

Google Cloud provides multiple services for distributed data processing

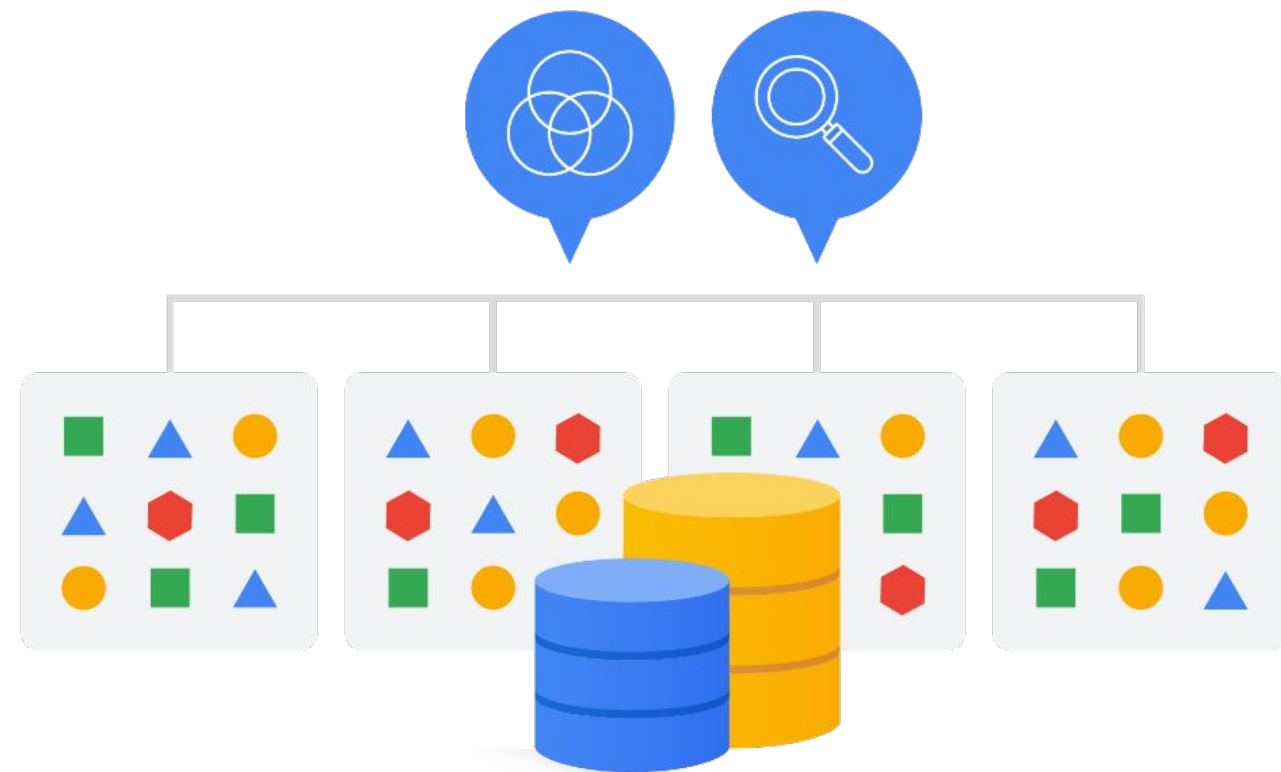


M5 — the escape hatch, not the destination

- Default stays ELT in BigQuery— reach beyond SQL only for custom code, streaming, non-BQ targets
- Batch vs streaming— a latency decision, not a religion; nightly covers most warehouse work
- Pub/Sub— the managed messaging backbone; your Service Broker / queue analogue
- Dataflow— managed Apache Beam runner for batch and stream; templates = configure, not code
- Depth lives elsewhere— the Spark lab is optional homework; streaming is Course 4

Conceptual pass only — know the names and when to reach for each.

Batch processing versus streaming data processing

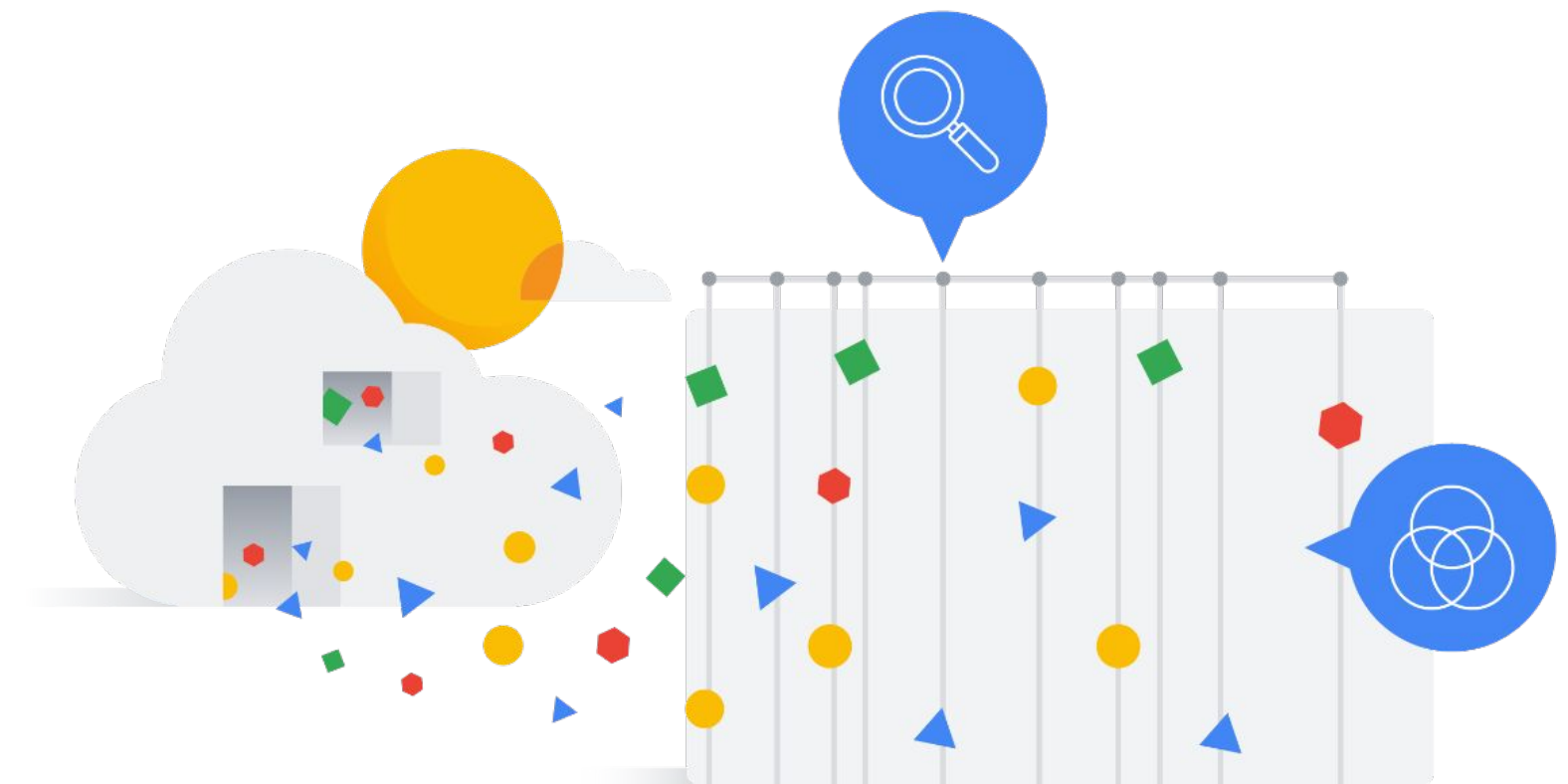


Batch processing

Processing and analysis happens on a set of stored data.

Payroll system

Billing system



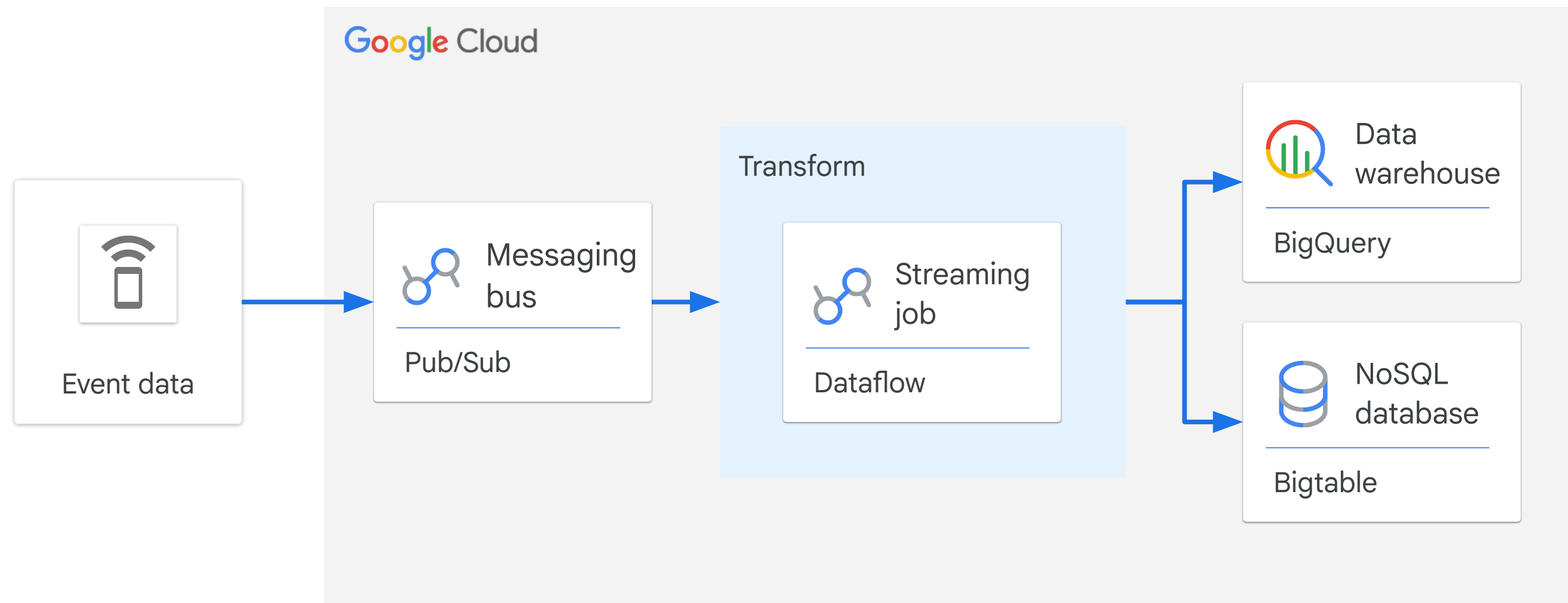
Streaming data processing

A flow of data records generated by various data sources.

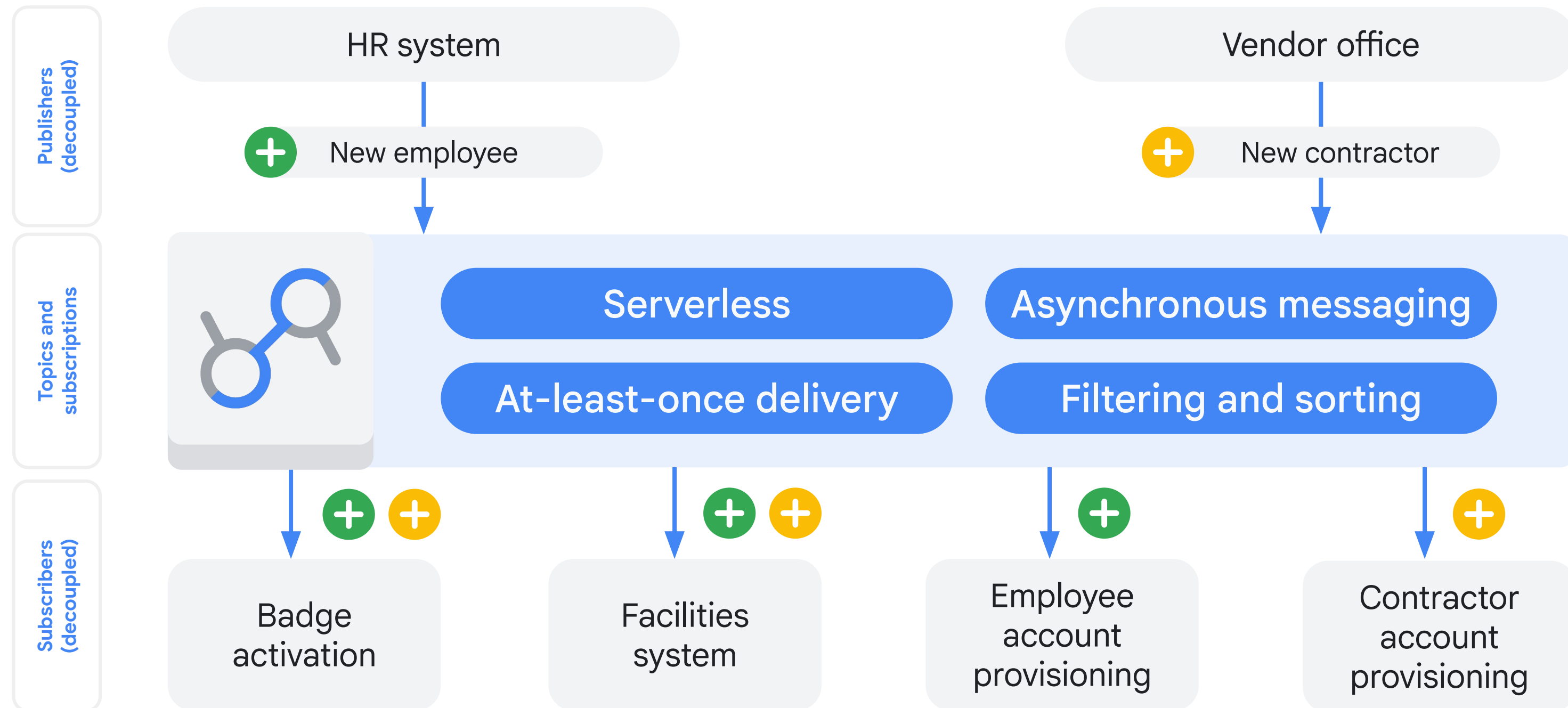
Fraud detection

Intrusion detection

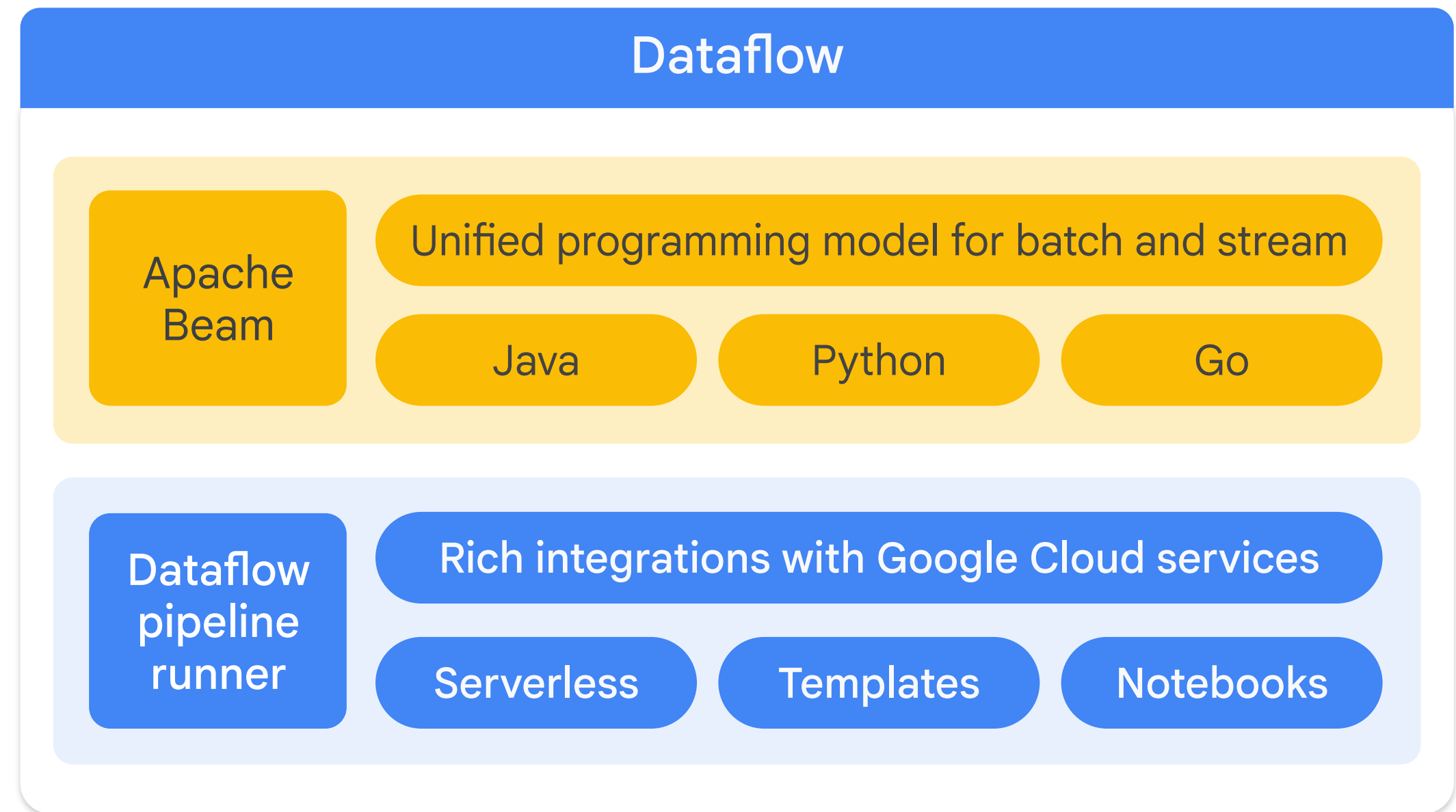
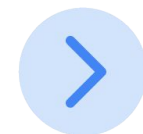
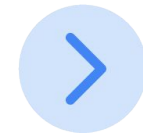
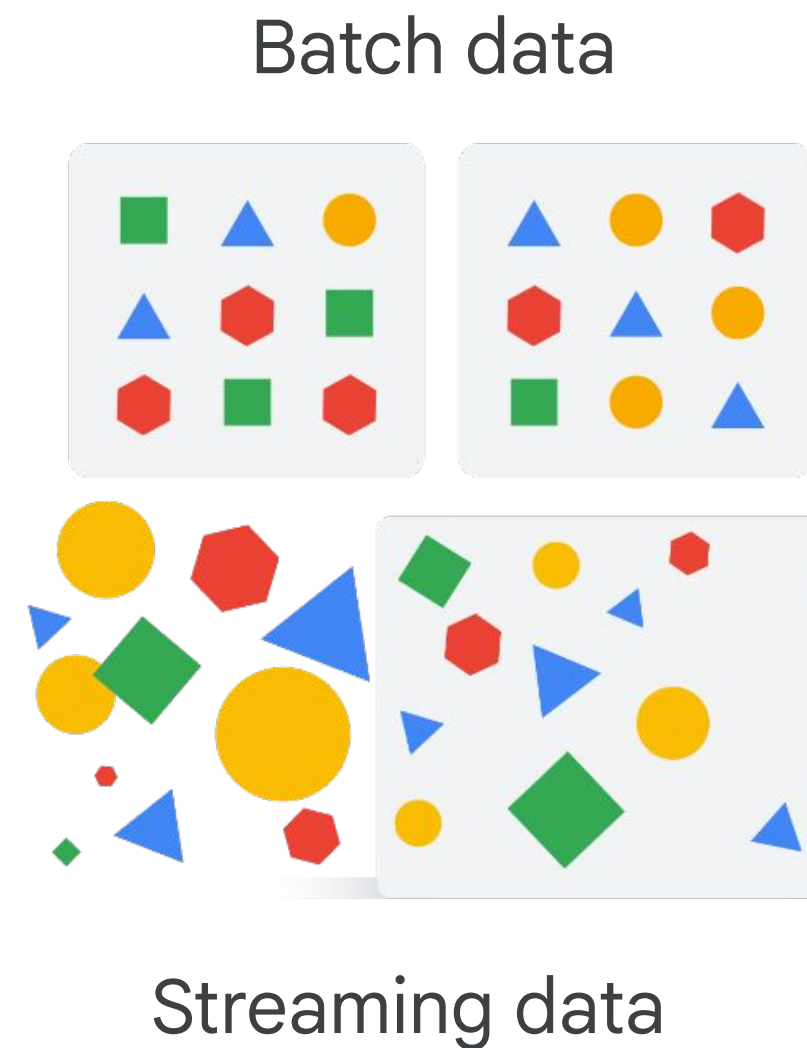
Streaming ETL workflows require continuous data ingestion, processing, and near real-time analytics



Use Pub/Sub to ingest high volumes of event data and distribute to different consuming systems



Dataflow can process both batch and streaming data using the Apache Beam programming model



Example: Streaming and transforming messages from Pub/Sub to BigQuery

```
with beam.Pipeline(options=pipeline_options) as pipeline:

    # Read messages from Pub/Sub
    messages = pipeline | 'read' >> ReadFromPubSub(topic='<PUBSUB_TOPIC>')

    # Parse messages
    def parse_message(message):
        return json.loads(message.decode('utf-8'))

    parsed_messages = messages | 'parse' >> beam.Map(parse_message)

    # Write parsed messages to BigQuery
    parsed_messages | 'write' >> WriteToBigQuery(
        table='<BIGQUERY_TABLE>',
        schema=table_schema,
        create_disposition=BigQueryDisposition.CREATE_IF_NEEDED,
        write_disposition=BigQueryDisposition.WRITE_APPEND
    )

    # Run the pipeline
    pipeline.run()
```

ReadFromPubSub():

- Retrieves messages from a Pub/Sub topic or subscription.

beam.Map():

- Applies a specified transformation to the message.

WriteToBigQuery():

- Writes the transformed message into a BigQuery table.

Use Dataflow templates for recurring pipeline executions with different parameters

 **CREATE JOB FROM TEMPLATE**

Dataflow template
Cloud Storage Text to BigQuery (Stream) ▼

Source
gs:// Cloud Storage Input File(s) [BROWSE](#)

Target
BigQuery output table [BROWSE](#)

Required Parameters ▼

Benefits of using templates

- Pipeline design is separate from deployment.
- Parameters customize the pipeline and make it reusable with different inputs.
- Templated pipelines can be deployed from the Google Cloud console, command-line interface, or API.
- Use Google-provided pre-built templates for common scenarios.



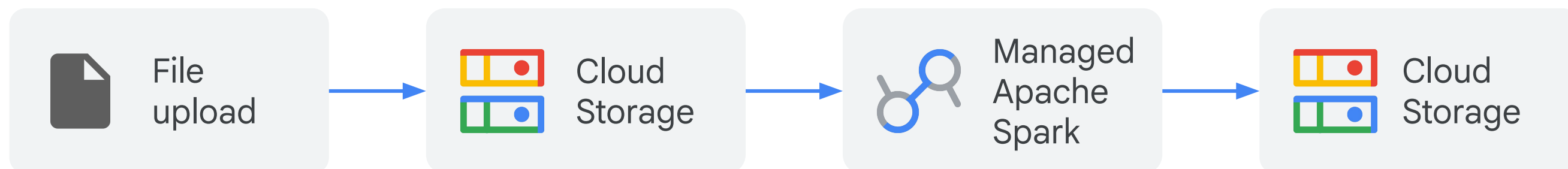
You can create your own custom templates.

ELT and ETL workloads can be automated to run on a recurring basis

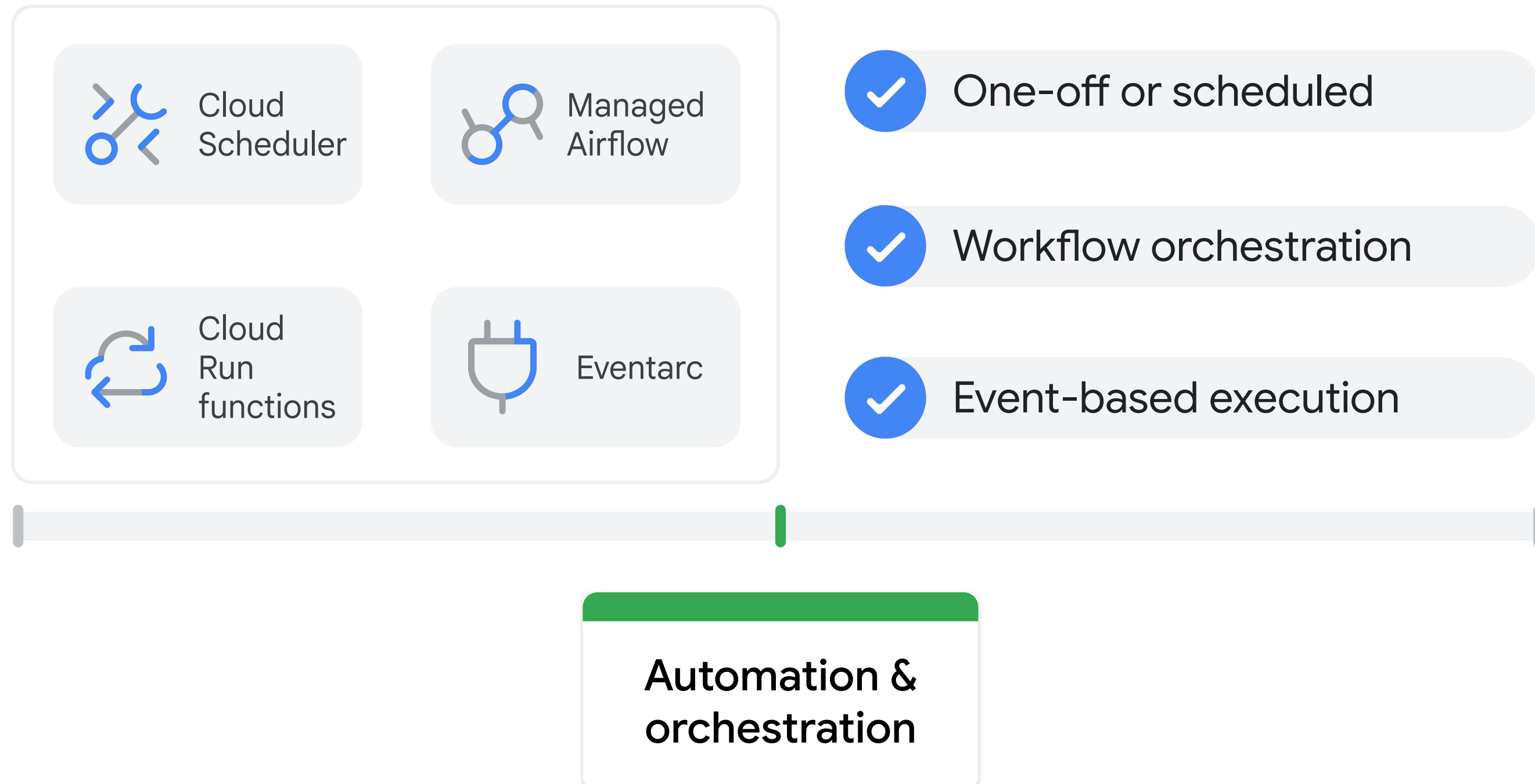
Scheduled ELT example



Event-driven ETL batch example



Google Cloud provides multiple services for automating and orchestrating your workloads

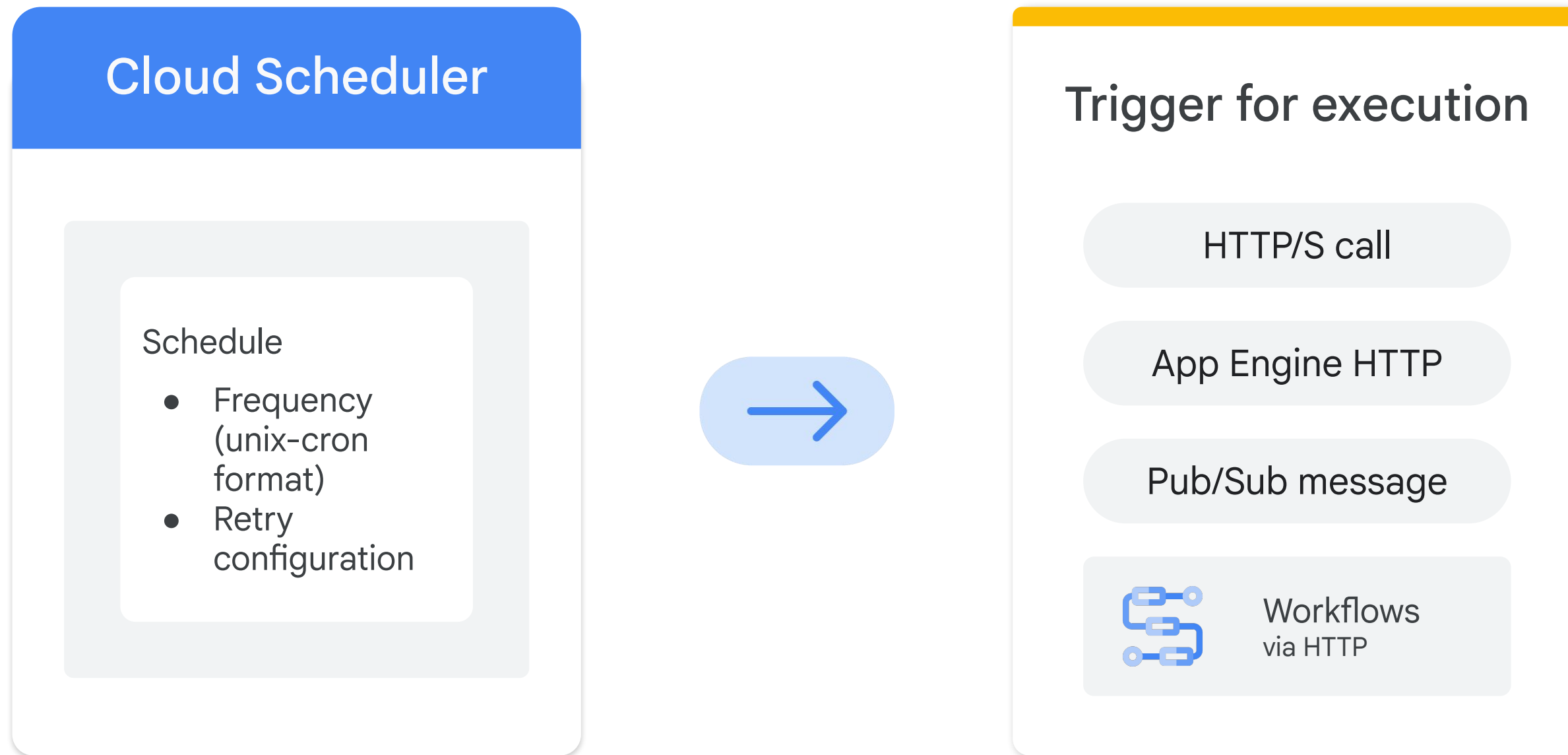


M6 — for the M&O team: who keeps the jobs running

- Your instincts translate— SQL Agent schedules and master packages, one for one
- Cloud Scheduler— managed cron triggering HTTP or Pub/Sub; no scheduler VM to patch
- Scheduled queries— one SQL statement on a timer; the closing demo puts one live
- Dataform release configs— your Silver→Gold runs on a cadence, compiled from a git commit
- Composer (managed Airflow)— the orchestrator when many pipelines depend on each other
- Eventarc + Cloud Run functions— event triggers and glue code, your file-watcher analogue

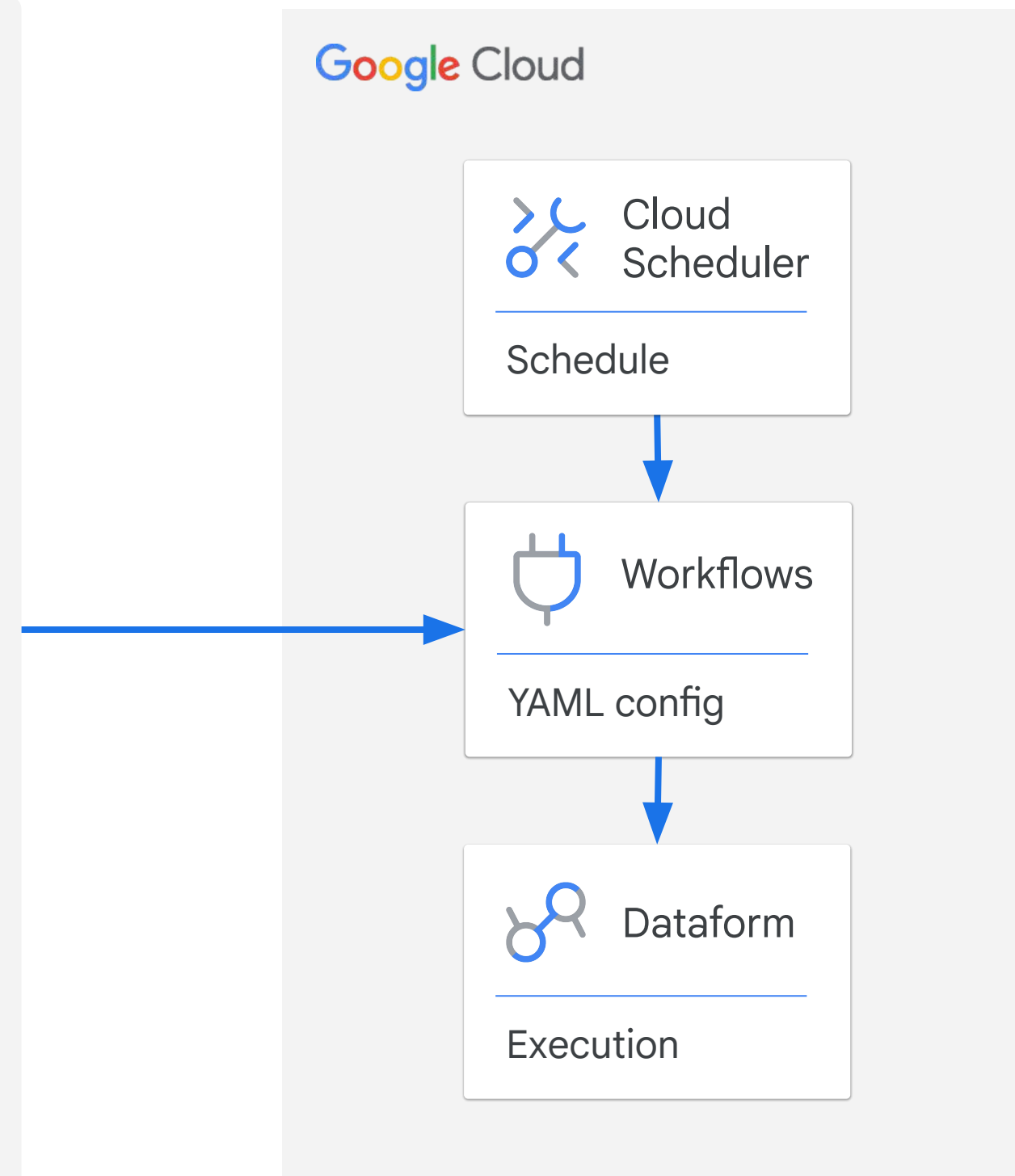
Your discipline is unchanged — source control, review, staged deployment, monitoring. Only the buttons move.

Cloud Scheduler invokes your workloads at recurring intervals



Example: Trigger a Dataform SQL workflow

```
- createCompilationResult:
  call: http.post
  args:
    url: ${"https://dataform.googleapis.com/[...]"}
    auth:
      type: OAuth2
    body:
      gitCommitish: <your_branch>
  result: compilationResult
- createWorkflowInvocation:
  call: http.post
  args:
    url: ${"https://dataform.googleapis.com/[...]"}
    auth:
      type: OAuth2
    body:
      compilationResult: ${compilationResult.body.name}
      invocationConfig:
        includedTags:
          - <your_tag>
  result: workflowInvocation
```



Compare automation options

	Cloud Scheduler	Managed Airflow	Cloud Run functions	Eventarc
Trigger type	schedule, manual	schedule, manual	event	event
Serverless	yes	no	yes	yes
Coding effort	low	medium	high	high
Programming languages	YAML (with Workflows)	Python	Python, Java, Go, Node.js, Ruby, PHP, .NET core	any

Day 1 done — what you now have

- ✓ Your data lands in BigQuery— Datastream CDC killed the nightly extract window before lunch
- ✓ The loading toolbox— console, CLI, autodetect pitfalls, Parquet vs CSV, LOAD DATA in SQL
- ✓ The Silver→Gold transform— a working Dataform SQL workflow from this afternoon's lab
- ✓ The automation map— Scheduler $\approx$ SQL Agent, Composer $\approx$ orchestrator, releases compiled from git
- ✓ Tomorrow— lakehouse and the medallion architecture: Bronze/Silver/Gold made formal, then Power BI